

```
+++ date = '2025-10-26T00:55:11+08:00' draft = false title = 'Windows内核笔记-保护模式' description = '滴水中级班保护模式笔记，记录了一些碎片知识以及课程的课后习题' tags = ['逆向','内核','C/C++'] +++
```

前言

说是笔记，实际上还是课后习题的内容，我不擅长写笔记，就不误导别人了

常见问题

这个写在前面是便于错误速查，若有同样的问题可以直接在这里速查，免得去翻后面的内容

这里对于错误的定义是不符合视频内容的东西，比如说软件问题，命令报错等等

所用镜像和软件版本

- WDK7600
- WinXP SP3
 - zh-hans_windows_xp_professional_with_service_pack_3_x86_cd_vl_x14-74070.iso
 - SHA1: D142469D0C3953D8E4A6A490A58052EF52837F0F
 - 601.04MB
 - 2008-05-02
 - 下载地址: ed2k://file|zh-hans_windows_xp_professional_with_service_pack_3_x86_cd_vl_x14-74070.iso|630237184|EC51916C9D9B8B931195EE0D6EE9B40E|/
 - 序列号: MRX3F-47B9T-2487J-KWKMF-RPWBV

VC6

1. 编译前要清理

如果遇到以下错误，试试先清理项目再重新编译，构造调用门

```
Access violation - code c0000005 (!!! second chance !!!)
nt!NtWaitForDebugEvent+0x122:
8063a3aa f6412c05 test byte ptr [ecx+2Ch],5
```

Windbg

1. !process 0 0符号错误

```
kd> !process 0 0
**** NT ACTIVE PROCESS DUMP ****
NT symbols are incorrect, please fix symbols
```

解决方法:

```
.sympath SRV*C:\Symbols*https://msdl.microsoft.com/download/symbols ;表示把本地缓存
放在 C:\Symbols, 并从微软符号服务器下载
.symfix          ; (可选) 自动设置到 MS 符号服务器
!sym noisy       ; 开启符号加载的详细信息 (便于排错)
.reload /f       ; 强制重载全部模块符号
.sympath         ; 查看当前符号路径
!mv m nt         ; 或 !mv m ntkrnl* 查看内核模块及其符号加载状态
!process 0 0     ; 重试
```

调用门实验蓝屏,C05等错误

- **参考资料** [进入0环蓝屏问题\(幽灵熔断\)](#)

【看雪】调用门的使用以及调试调用门的注意事项

[cc12138博客](#)

- **具体错误**

我并没有遇到[幽灵熔断](#)导致的蓝屏问题, 但我遇到了C05的错误, 我的代码如下

```
#include <windows.h>
#include <stdio.h>
DWORD x = 0;
DWORD y = 0;
DWORD z = 0;
void __declspec(naked) Func()
{
    __asm
    {
        int 3;
        pushad;
        pushfd;
        mov eax,[esp+0x24+0x8+0x8];
        mov dword ptr ds:[x],eax;
        mov eax,[esp+0x24+0x8+0x4];
        mov dword ptr ds:[y],eax;
        mov eax,[esp+0x24+0x8];
        mov dword ptr ds:[z],eax;
    }

    __asm
    {
        popfd;
        popad;
        retf 0xc; // push了3个参数, 需要手动平衡堆栈
    }
}
void PrintRegisetr()
{
    printf("%x %x %x",x, y, z);
```

```

}
int main()
{
    char buf[6];
    *(DWORD*) &buf[0] = 0x12345678;
    *(WORD*) &buf[4] = 0x48;
    __asm
    {
        push 3;
        push 2;
        push 1;
        call fword ptr [buf];
    }
    PrintRegisetr();
    return 0;
}

```

在阅读上面的资料后，所有线索指向内核断点会修改FS

要注意的是调用门最好不要步入(Step In)，步过(Step Over)最好。需要解释一下：

步入之后会修改FS寄存器的值为0x30

如果调用门中使用了参数，那么在RETF指令之后会直接蓝屏

如果没有使用参数，那么RETF指令返回到Ring3之后虽然不会蓝屏但是也会把FS寄存器的值置为0。。。原因我也不清楚，貌似是和中断有关。困扰了我好久也依然不知道原因。但是步过的话就一切正常。

这里我就直接步入了。注意：重启之后需要重新安装调用门。

注意注意：

从三环进入零环，或者从零环进入三环，fs都得用户自己手动来切换，如果不切换，可能会崩溃

• 解决办法

1. 不在内核产生断点
2. 使用push fs;pop fs(失败) 这个方法我没有成功，在内核产生断点依旧会修改FS的值为30，并且出内核后FS的值会恢复为0

段描述符与段选择子

笔记

1. GDT(全局描述符表) LDT(局部描述符表)

当我们执行类似MOV DS,AX指令时，CPU会查表，根据AX的值来决定查找GDT还是LDT,查找表的什么位置，查出多少数据。

也就是说，在执行MOV DS,AX时AX的值不能是随便的

可以用Windbg的

`r gdtr`(`r`是查看寄存器, `gdtr`是48位的, 不是96位的)来查找GDT表的地址

`r gdtl`来查看GDT表的长度

`dd 8003f000`查看地址为`8003f000`的数据, 以四字节分组显示(`dq`则是8字节分组显示) `dq 8003f000 L40`显示40组数据, 每组8字节, 从`8003f000`开始

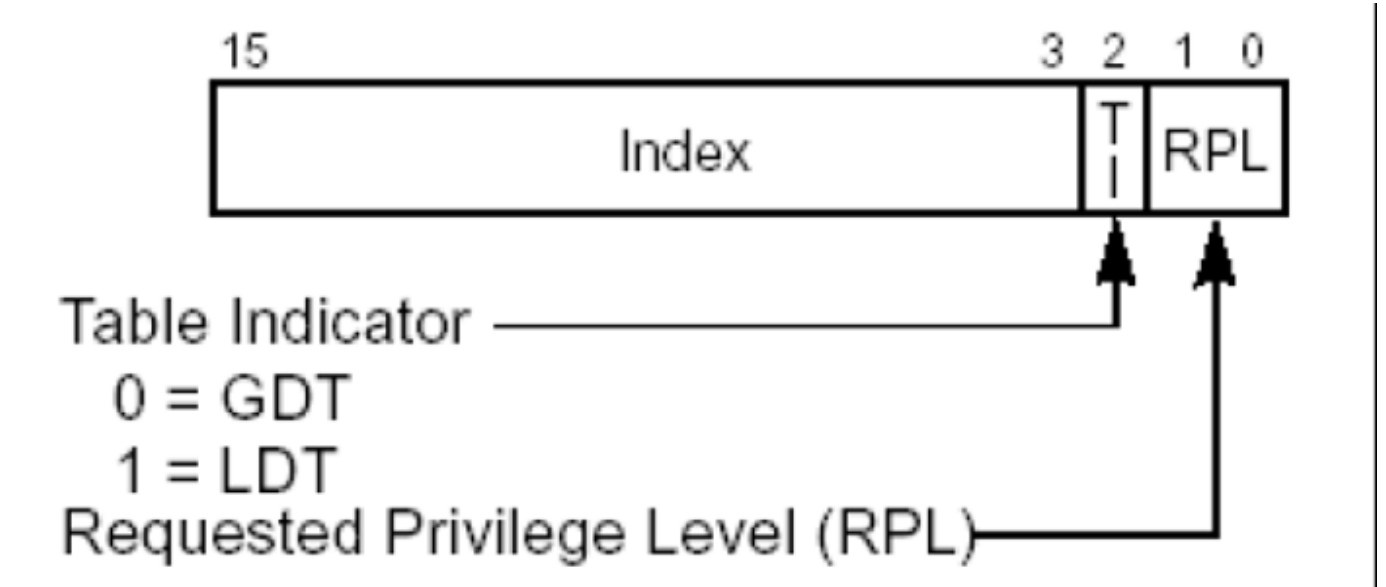
```
kd> r gdtr
gdtr=8003f000
kd> r gdtl
gdtl=000003ff
kd> dd 8003f000
8003f000  00000000 00000000 0000ffff 00cf9b00
8003f010  0000ffff 00cf9300 0000ffff 00cffb00
8003f020  0000ffff 00cff300 200020ab 80008b04
8003f030  f0000001 ffc093df 00000fff 0040f300
8003f040  0400ffff 0000f200 00000000 00000000
8003f050  af000068 80008954 af680068 80008954
8003f060  2f40ffff 00009302 80003fff 0000920b
8003f070  700003ff ff0092ff 0000ffff 80009a40
kd> dq 8003f000
8003f000  00000000`00000000 00cf9b00`0000ffff
8003f010  00cf9300`0000ffff 00cffb00`0000ffff
8003f020  00cff300`0000ffff 80008b04`200020ab
8003f030  ffc093df`f0000001 0040f300`0000ffff
8003f040  0000f200`0400ffff 00000000`00000000
8003f050  80008954`af000068 80008954`af680068
8003f060  00009302`2f40ffff 0000920b`80003fff
8003f070  ff0092ff`700003ff 80009a40`0000ffff
```

```
8003f000  00000000`00000000 00cf9b00`0000ffff
8003f010  00cf9300`0000ffff 00cffb00`0000ffff
8003f020  00cff300`0000ffff 80008b04`200020ab
8003f030  ffc093df`f0000001 0040f300`0000ffff
8003f040  0000f200`0400ffff 00000000`00000000
8003f050  80008954`af000068 80008954`af680068
8003f060  00009302`2f40ffff 0000920b`80003fff
8003f070  ff0092ff`700003ff 80009a40`0000ffff
```

以8字节显示是因为GDT表的每个项占8字节, 而GDT表的起始地址是`0x00000000`, 接下来分析每个项的含义

2. 段选择子

段选择子是一个16位的段描述符, 该描述符指向了定义该段的段描述符



RPL：请求特权级别

TI:

TI=0 查GDT表 TI=1 查LDT表

Index:

处理器将索引值乘以8 在加上GDT或者LDT的 基地址，就是要加载的 段描述符

例如拆分1B

```
1B
0000 0000 0001 1011
00000000000011 0 11
```

TI=0, 查GDT表

index=3, 查GDT的第三项(00cffb00`0000ffff)

3. 段描述符

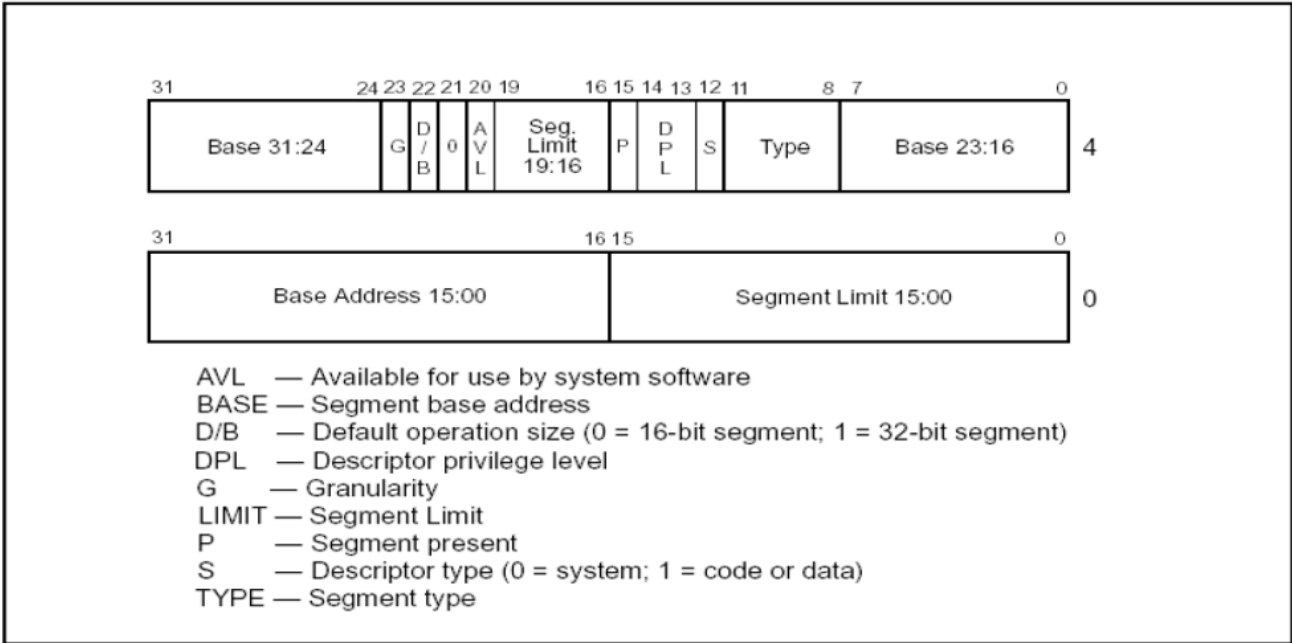


Figure 3-8. Segment Descriptor

4. 加载段描述符至段寄存器

除了MOV指令，我们还可以使用LES、LSS、LDS、LFS、LGS指令修改寄存器。

CS不能通过上述的指令进行修改，CS为代码段，CS的改变会导致EIP的改变，要改CS，必须要保证CS与EIP一起改，后面会讲。

```
char buffer[6];
__asm
{
    les ecx,fword ptr ds:[buffer] //高2个字节给es，低四个字节给ecx
}
```

注意：RPL<=DPL(在数值上)

课后练习

- 1、记住段描述符与段选择子的结构
- 2、使用LES、LDS等指令修改段寄存器

思考题：

段描述符共有64位，但需要填充的是80位，怎么填？

段描述符属性

笔记

域简称	全称	含义
S	系统 (System)	S=0代表该描述符是一个系统段，S=1代表该描述符是代码段，数据段或者堆栈段
P	存在 (Present)	指示该段是否存在于内存中。如果P=0，则当将指向段描述符的段选择器加载到段寄存器中时，处理器将生成段不存在异常。内存管理软件可以使用此标志来控制哪些段实际加载到物理内存中。它除了提供分页外，还提供了一个管理虚拟内存的控件。
DPL	描述符特权级 (Descriptor Privilege Level)	这两位定义了该段的特权级别（0~3），简单来说，仅当要访问该段的程序的特权级别（CPL）等于或高于这个段的级时CPU才允许其访问，否则便会抛出保护性异常（GPF）
D/B	Default/Big	对于代码段，该位表示的是这个代码段默认的位数(Default Bit)。D=0：16位代码段 D=1：32位代码段 对于栈段，该位称为B(Big)标志。B=1：使用32位堆栈指针（保存在ESP中） B=0：使用16位堆栈指针（保存在SP中） 对于数据段，该位称为B(Big)标志，指定了段的上界。B=1：段的上界是0xFFFFFFFF(4GB) B=0：段的上界是0xFFFF(64KB)
G	Granularity	该位用于描述段界限的粒度。G=1：段界限是4K字节对齐的。G=0：段界限是字节对齐的。
Type	段类型	由S位来决定段类型，具体内容见下面的分析
L	64-bit代码段	用于描述IA-32e模式下的代码段。L=1：代码段包含64位代码 L=0：代码段包含兼容模型的代码
AVL	Available and reserved bits	供系统软件（操作系统）使用

• **P位**

P位在段描述符的高4字节第15位，用于描述段是否有效

P=1 段描述符有效

P=0 段描述符无效

当我们通过指令往段寄存器加载段描述符时，CPU首先检查P位，若P位为0，则不做后续检查

• **G位**

G位在段描述符的高4字节第23位

• **段描述符与段寄存器的关系**

段寄存器的结构如下



```
WORD Selector;    //16位
WORD Attribute;   //16位
DWORD Base;       //32位
DWORD Limit;      //32位
```

按照段描述符来拆分段寄存器

- **Attribute**

段描述符高4字节第8位至24位

- **Base**

Base被拆分为多块，来自段描述符的

1. (Base 31:24)高4字节24位到31位
2. (Base 23:16)高4字节0位到7位
3. (Base 15:0)低4字节16位到31位

- **Limit**

Limit被拆分为多块，来自段描述符的

1. (Limit 19:16)高4字节16位到19位
2. (Limit 15:0)低4字节0位到15位

但是这样加起来只有20位(FFFFF)，剩下的12位在哪？

若G=1，则Limit为4K字节对齐的段界限(FFFFFFF)，若G=0，则Limit为字节对齐的段界限(000FFFFF)

- **S位**

S = 1 代码段或者数据段描述符

S = 0 系统段描述符

S位，DPL，P位构成一个字，其中在windows操作系统中DPL只有00或11两种可能

所以当我们想分析代码段或数据段TYPE域属性，S位一定是1，P位一定是1，则只需要注意段描述符高4字节的第5个字若为0x9或0xF，则是代码段或数据段


```
kd> r gdtr
gdtr=8003f000
kd> dq 8003f000
8003f000  00000000\00000000 00cf9b00\0000ffff
8003f010  00cf9300\0000ffff 00cffb00\0000ffff
8003f020  00cff300\0000ffff 80008b04\200020ab
8003f030  ffc093df\f0000001 0040f300\00000fff
8003f040  0000f200\0400ffff 00000000\00000000
8003f050  80008954\af000068 80008954\af680068
8003f060  00009302\2f40ffff 0000920b\80003fff
8003f070  ff0092ff\700003ff 80009a40\0000ffff
```

若要再细分是代码段还是数据段，则需要分析TYPE域

• TYPE域

1. S位为1

Type Field					Descriptor Type	Description
Decimal	11	10 E	9 W	8 A		
0	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
2	0	0	1	0	Data	Read/Write
3	0	0	1	1	Data	Read/Write, accessed
4	0	1	0	0	Data	Read-Only, expand-down
5	0	1	0	1	Data	Read-Only, expand-down, accessed
6	0	1	1	0	Data	Read/Write, expand-down
7	0	1	1	1	Data	Read/Write, expand-down, accessed
		C	R	A		
8	1	0	0	0	Code	Execute-Only
9	1	0	0	1	Code	Execute-Only, accessed
10	1	0	1	0	Code	Execute/Read
11	1	0	1	1	Code	Execute/Read, accessed
12	1	1	0	0	Code	Execute-Only, conforming
13	1	1	0	1	Code	Execute-Only, conforming, accessed
14	1	1	1	0	Code	Execute/Read-Only, conforming
15	1	1	1	1	Code	Execute/Read-Only, conforming, accessed

TYPE域位于段描述符高4字节第6字，如上图所示，若大于等于0x8，则是代码段，否则数据段

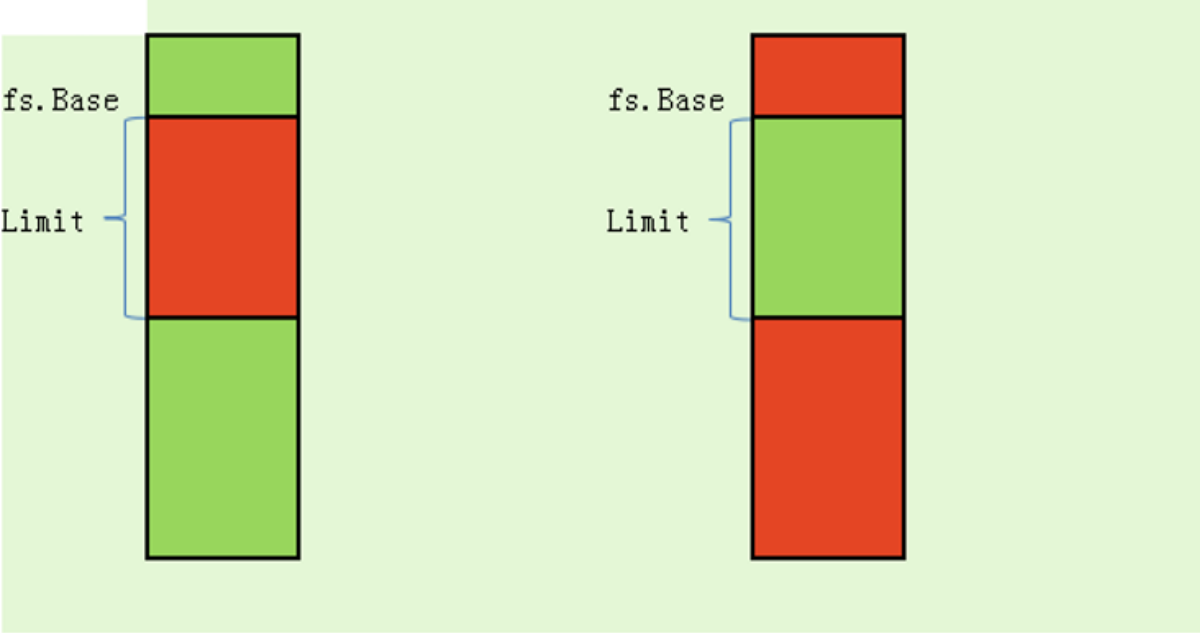
◦ 数据段

A 访问位，表示该位最后一次被操作系统清零后，该段是否被访问过.每当处理器将该段选择符置入某个段寄存器时，就将该位置1.

W 是否可写

E 扩展方向(为1则向下拓展)

如下图，红色部分是有效的，图一是向上拓展，图二向下拓展



◦ 代码段

A 访问位

R 可读位

C 一致位

C = 1 一致代码段

C = 0 非一致代码段

2. S位为0

当S=0时，该段描述符为系统描述符，系统描述符有分为以下类型

Table 3-2. System-Segment and Gate-Descriptor Types

Type Field					Description
Decimal	11	10	9	8	
0	0	0	0	0	Reserved
1	0	0	0	1	16-Bit TSS (Available)
2	0	0	1	0	LDT
3	0	0	1	1	16-Bit TSS (Busy)
4	0	1	0	0	16-Bit Call Gate
5	0	1	0	1	Task Gate
6	0	1	1	0	16-Bit Interrupt Gate
7	0	1	1	1	16-Bit Trap Gate
8	1	0	0	0	Reserved
9	1	0	0	1	32-Bit TSS (Available)
10	1	0	1	0	Reserved
11	1	0	1	1	32-Bit TSS (Busy)
12	1	1	0	0	32-Bit Call Gate
13	1	1	0	1	Reserved
14	1	1	1	0	32-Bit Interrupt Gate
15	1	1	1	1	32-Bit Trap Gate

- **D/B位**

位于段描述符的第22位

情况一：对CS段的影响

D = 1 采用32位寻址方式

D = 0 采用16位寻址方式

前缀67 改变寻址方式

情况二：对SS段的影响

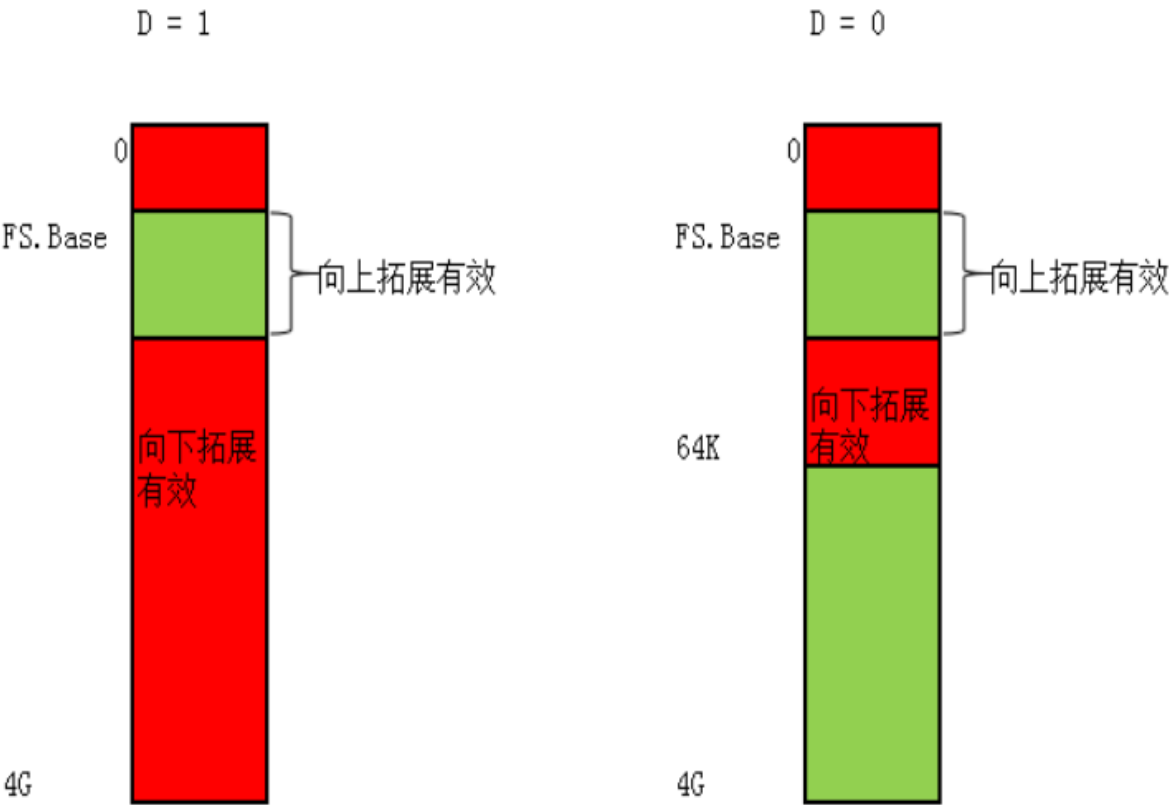
D = 1 隐式堆栈访问指令（如：PUSH POP CALL） 使用32位堆栈指针寄存器ESP

D = 0 隐式堆栈访问指令（如：PUSH POP CALL） 使用16位堆栈指针寄存器SP

情况三：向下拓展的数据段

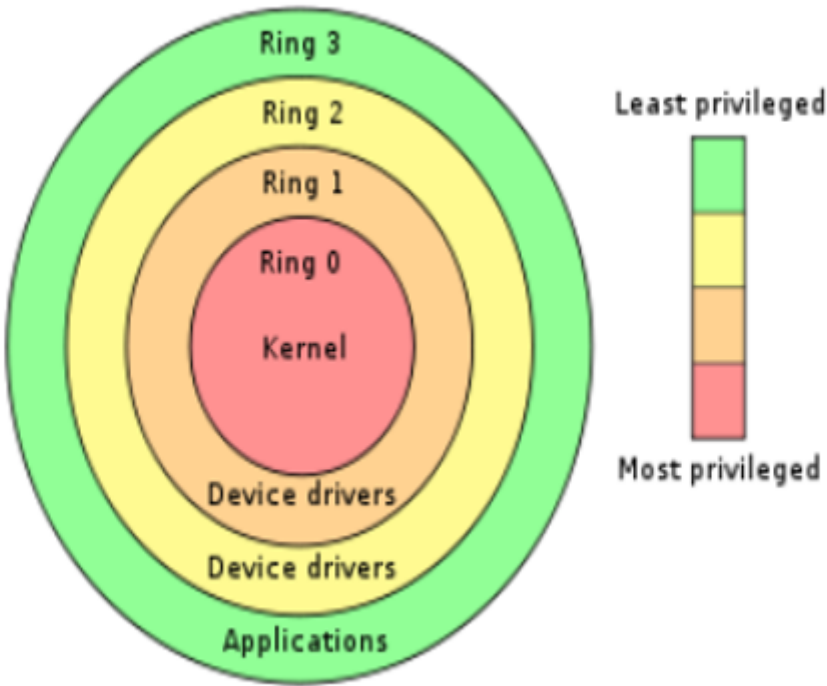
D = 1 段上线为4GB

D = 0 段上线为64KB



• 段权限检查

1. CPU分级



ring0权限最高，ring3权限最低

2. CPL(Current Privilege Level): 当前特权级

CPL用来查看当前程序属于几环

CS和SS中存储的段选择子后2位就是CPL

例如：CS: 001B = 0000 0000 0001 1011 最后两位加起来为3，则当前程序处于3环(ring3)

3. DPL(Descriptor Privilege Level): 描述符特权级别

DPL存储在段描述符中，规定了访问该段所需要的特权级别是什么。通俗的理解：如果你想访问我，那么你应该具备什么特权

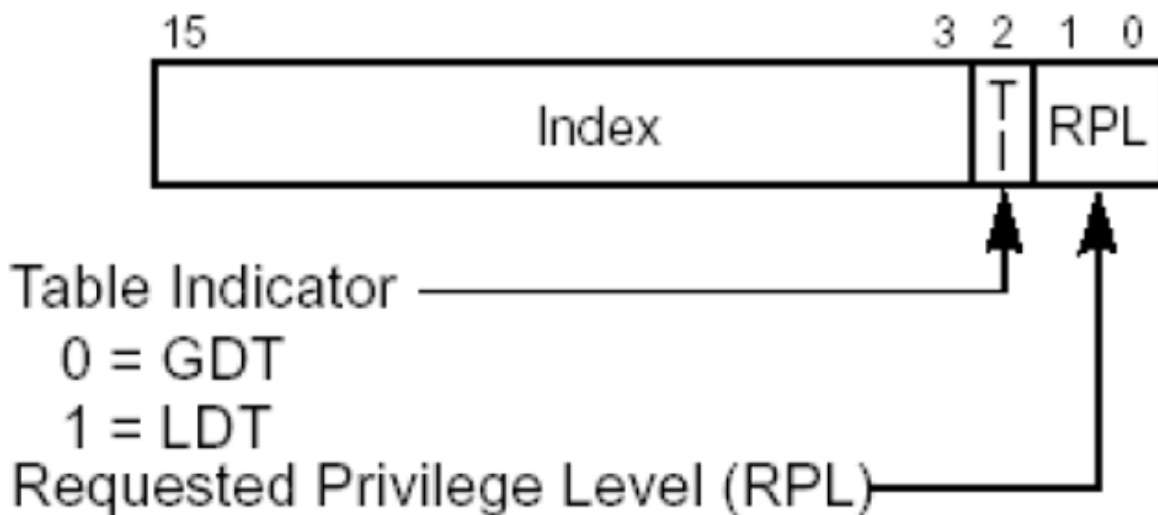
举例说明：

```
mov DS,AX
```

如果AX指向的段DPL = 0 但当前程序的CPL = 3 这行指令是不会成功的！

4. RPL(Request Privilege Level): 请求特权级别

RPL是针对段选择子而言的，每个段的选子都有自己的RPL。



举例说明：

```
Mov ax,0008 与 Mov ax,000B //段选择子
Mov ds,ax     Mov ds,ax     //将段描述
```

0008 = 1 0 00

000B = 1 0 11

指向的是同一个段描述符,但RPL是不一样的.

5. 数据段的权限检查

参考如下代码：

比如当前程序处于0环，也就是说CPL=0

```
Mov ax,000B //1011    RPL = 3
Mov ds,ax    //ax指向的段描述符的DPL = 0
```

数据段的权限检查：

CPL <= DPL 并且 RPL <= DPL (数值上的比较)

注意：

代码段和系统段描述符中的检查方式并不一样

6. 总结：

CPL: CPU当前的权限级别

DPL: 如果你想访问我，你应该具备什么样的权限

RPL: 用什么权限去访问一个段

为啥要有RPL?

我们本可以用"读 写"的权限去打开一个文件，但为了避免出错，有些时候我们使用"只读"的权限去打开。

课后练习

- 分析段选择子为0x1B、0x23对应的段描述符，并将内容填写到段寄存器结构体中

1. 0x1B

```
1B
0000 0000 0001 1011
0000000000011 0 11
```

index=3, 查GDT的第三项(00cffb00`0000ffff)

```
00cffb00`0000ffff
0000 0000 1100 1111 1111 1011 0000 0000
0000 0000 0000 0000 1111 1111 1111 1111
Selector = 0x1B
Attribute = 1100 1111 1111 1011 = 0xCFFB
Base = 0000 0000 0000 0000 0000 0000 0000 0000 = 0x00000000
Limit = 1111 1111 1111 1111 = 0xFFFFFFFF(G=1)
```

2. 0x23

```
23
0000 0000 0010 0011
0000000000100 0 11
```

index=4, 查GDT的第五项(00cff300`0000ffff)

```
00cff300`0000ffff
0000 0000 1100 1111 1111 0011 0000 0000
0000 0000 0000 0000 1111 1111 1111 1111
Selector = 0x23
Attribute = 1100 1111 1111 0011 = 0xCFF3
Base = 0000 0000 0000 0000 0000 0000 0000 0000 = 0x00000000
Limit = 1111 1111 1111 1111 = 0xFFFFFFFF(G=1)
```

代码间的跳转

笔记

- 段间跳转

```
JMP 0x20:0x004183D7 ;CPU如何执行这行代码?
```

(1) 段选择子拆分

0x20 对应二进制形式 0000 0000 0010 0000

RPL = 00

TI = 0

Index = 4

(2) 查表得到段描述符

TI = 0 所以查GDT表

Index = 4 找到对应的段描述符

四种情况可以跳转：代码段、调用门、TSS任务段、任务门

(3) 权限检查

如果是非一致代码段，要求： $CPL == DPL$ 并且 $RPL \leq DPL$

如果是一致代码段，要求： $CPL \geq DPL$

(4) 加载段描述符

通过上面的权限检查后，CPU会将段描述符加载到CS段寄存器中。

(5) 代码执行

CPU将 $CS.Base + Offset$ 的值写入EIP 然后执行CS:EIP处的代码，段间跳转结束。

课后练习

1. 实现段间跳转

[看雪-保护模式学习笔记之段机制](#)

长调用与短调用

笔记

• 短调用

CALL 立即数/寄存器/内存

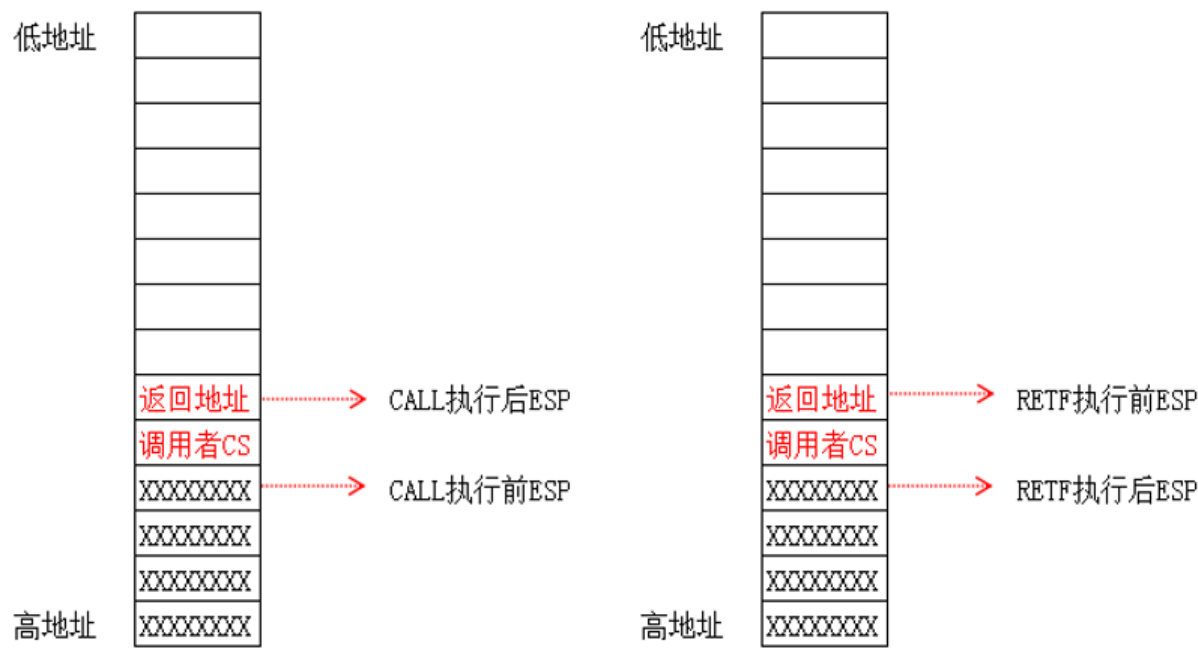
发生改变的寄存器：ESP EIP



• 长调用(跨段不提权)


```
CALL CS:EIP(EIP是废弃的)
```

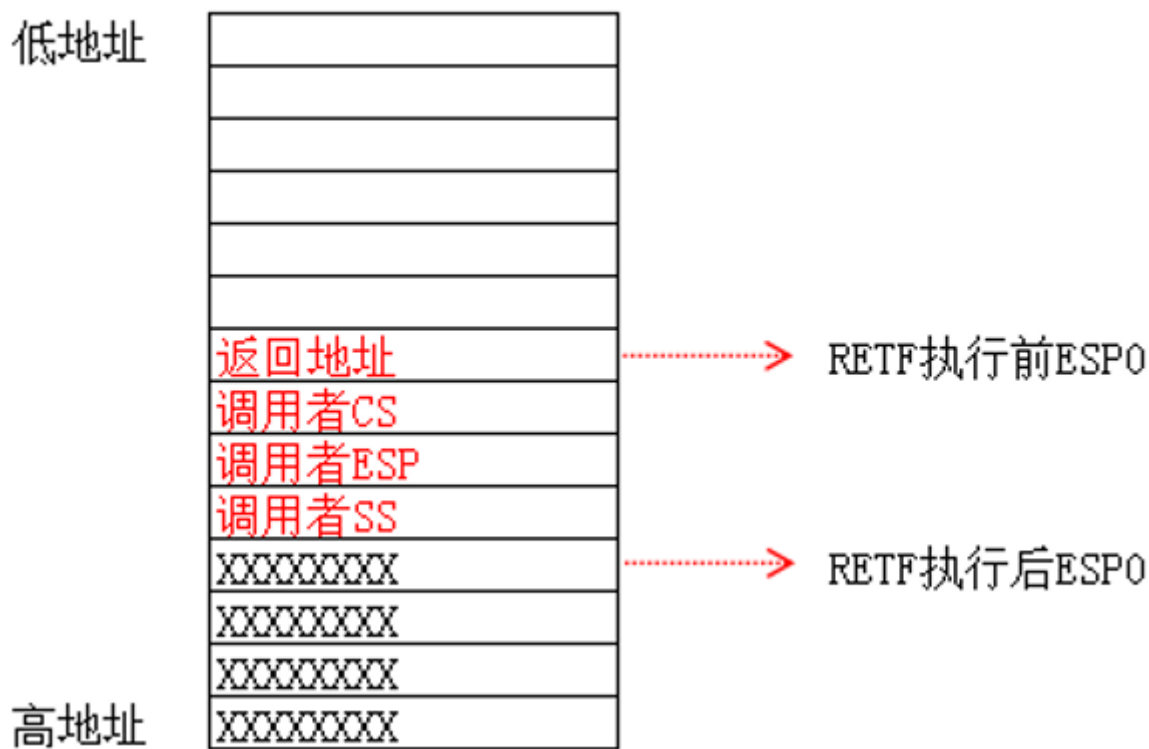
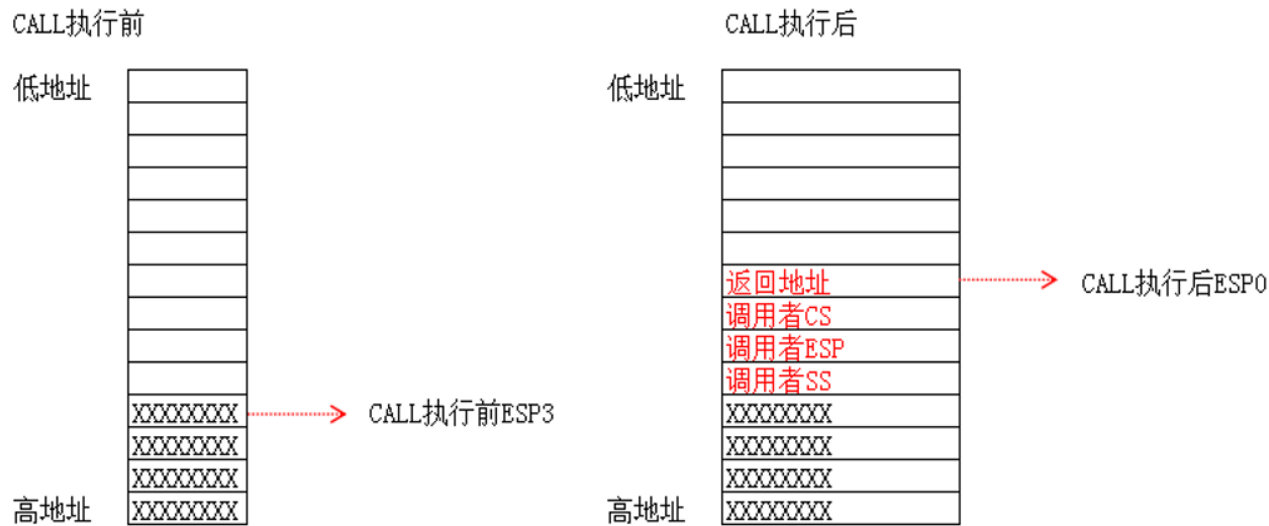
发生改变的寄存器：**ESP EIP CS**



• **长调用(跨段提权)**

```
CALL CS:EIP(EIP是废弃的)
```

发生改变的寄存器：**ESP EIP CS SS**



- 总结
 - 1. 跨段调用时，一旦有权限切换，就会切换堆栈.
 - 2. CS的权限一旦改变，SS的权限也要随着改变，CS与SS的等级必须一样.
 - 3. JMP FAR 只能跳转到同级非一致代码段，但CALL FAR可以通过调用门 提权，提升CPL的权限.
- SS与ESP从哪里来？ 参见TSS段.

调用门

笔记

1. 调用门执行流程

指令格式：CALL CS:EIP(EIP是废弃的，跳转地址由调用门描述符决定)

执行步骤：

1. 根据CS的值 查GDT表，找到对应的段描述符 这个描述符是一个调用门.
2. 在调用门描述符中存储另一个代码段段的选择子.
3. 选择子指向的段的 **段.Base + 偏移地址** 就是真正要执行的地址.

2. 门描述符

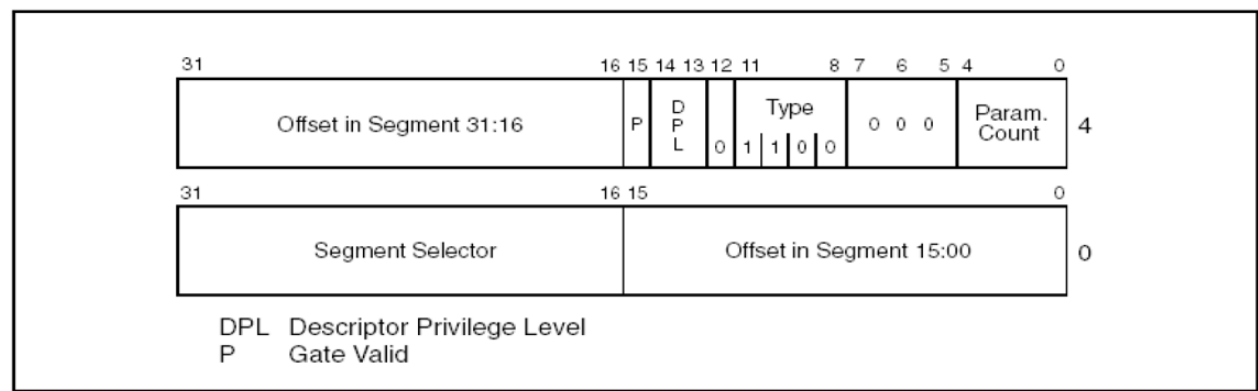


Figure 4-7. Call-Gate Descriptor

调用门描述符的P位,DPL都要为1, S位为0, TYPE域为1100

3. 构造一个调用门

根据门描述符来构造一个调用门，基础结构如下

```
struct CallGateDescriptor {
    unsigned short offset_low;      // 目标代码段偏移地址的低 16 位
    unsigned short selector;       // 目标代码段的段选择子
    unsigned char param_count : 5; // 参数数量
    unsigned char reserved : 3;    // 保留位 (通常为 0)
    unsigned char Type : 4;        // 属性 1110 (对应 32 位调用门)
    unsigned char s : 1;          // 0系统段 1数据段存储段
    unsigned char dpl : 2;        // 描述符特权级别 (DPL)
    unsigned char present : 1;     // 存在位 (P)
    unsigned short offset_high;    // 目标代码段偏移地址的高 16 位
};
```

其中**两段Offset相加则为跳转目标地址**,

4. 代码测试

步骤一：代码测试，并观察堆栈与寄存器的变化.

记录执行前的寄存器值：

CS SS ESP

步骤二：在测试代码中加入特权指令并读取高2G内存.

课后练习

1. 实现调用门(无参提权)

```
kd> r gdtr
gdtr=8003f000
kd> dq 8003f000
8003f000  00000000`00000000 00cf9b00`0000ffff
8003f010  00cf9300`0000ffff 00cffb00`0000ffff
8003f020  00cff300`0000ffff 80008b04`200020ab
8003f030  ffc093df`f0000001 0040f300`00000fff
8003f040  0000f200`0400ffff 00000000`00000000
8003f050  80008954`af000068 80008954`af680068
8003f060  00009302`2f40ffff 0000920b`80003fff
8003f070  ff0092ff`700003ff 80009a40`0000ffff
```

找到未被使用的段描述符，位置在第9个，修改为0000EC00`00080000`

```
eq 8003f048 0000EC00`00080000
```

此时跳转地址为0地址，下面的代码是一定会报错的

测试代码

```
#include <iostream>
#include <Windows.h>

void __declspec(naked) GetRegisetr()
{
    __asm
    {
        int 3
        retf
    }
}

int main()
{
    char buf[6];
    *(DWORD*) &buf[0] = 0x12345678;
    *(WORD*) &buf[4] = 0x48;
    __asm
    {
        call fword ptr[buf]
    }
}
```

```

    getchar();
    return 0;
}

```

运行后报错

```

watchdog!WdUpdateRecoveryState: Recovery enabled.
Access violation - code c0000005 (!!! second chance !!!)
nt!KiServiceExit2:
8053e7f0 fa cli

```

那如何通过调用门去调用代码所写的GetRegisetr函数？那就需要获取GetRegisetr的地址了

```

--- c:\vc6\msdev98\myprojects\test1\main.cpp
1:    #include <iostream>
2:    #include <Windows.h>
3:
4:    void __declspec(naked) GetRegisetr()
5:    {
00401030    int            3
6:        __asm
7:        {
8:            int 3
9:            retf
00401031    retf

```

在调试中获取了

GetRegisetr的地址为00401030，那么就修改调用门为

```
eq 8003f048 0040EC00`00081030
```

系统卡死，可以在Windbg里看到系统被断下来了，因为我们的GetRegisetr函数里写了int 3断点指令

现在回去观察一下寄存器

```

EAX = CCCCCCCC EBX = 7FFD9000 ECX = 00000000 EDX = 00380D78 ESI = 00000000
EDI = 0012FF80 EIP = 00401065 ESP = 0012FF28 EBP = 0012FF80 EFL = 00000212
MM0 = 0000000000000000 MM1 = 0000000000000000 MM2 = 0000000000000000
MM3 = 0000000000000000 MM4 = 0000000000000000 MM5 = 0000000000000000
MM6 = 0000000000000000 MM7 = 0000000000000000
XMM0 = 00000000000000000000000000000000 XMM1 = 00000000000000000000000000000000
XMM2 = 00000000000000000000000000000000 XMM3 = 00000000000000000000000000000000
XMM4 = 00000000000000000000000000000000 XMM5 = 00000000000000000000000000000000
XMM6 = 00000000000000000000000000000000 XMM7 = 00000000000000000000000000000000
CS = 001B DS = 0023 ES = 0023 SS = 0023 FS = 003B GS = 0000 OV=0 UP=0 EI=1 PL=0
ZR=0 AC=1 PE=0 CY=0

```

在执行调用门前，寄存器(长调用(跨段提权)所push的寄存器)为

```

CS: 0x1B
SS: 0x23
ESP: 0012FF28

```

cs	8
efl	212
esp	b168cdd0
ss	10

继续运行代码，在Windbg里断下

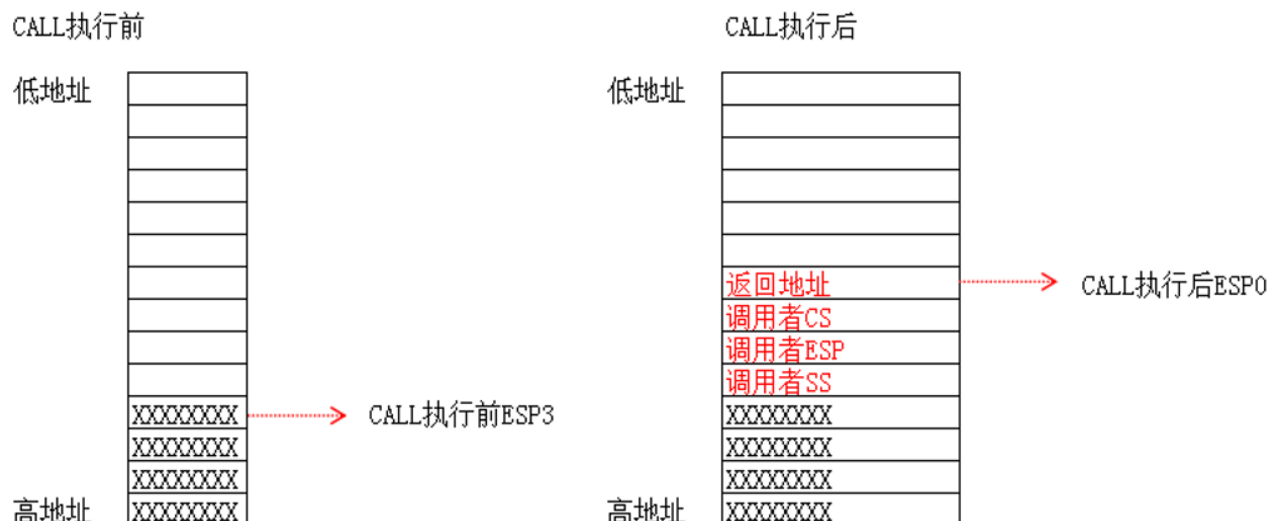
```
CS: 0x8
SS: 0x10
ESP: b168cdd0
```

可以看到ESP的地址远远大于8，已经进入0环了，再看看堆栈

[illegible]

```
00401068 0000001b 0012ff28 00000023
```

再看看之前长调用(跨段提权)的堆栈图



不难发现对应关系为

```
CS: 0x00000023
ESP: 0x0012ff28
SS: 0x0000001B
返回地址: 0x00401068
```

既然成功提权，那么就可以做一下只有内核能做的事了，比如读取高2G内存

```
#include <iostream>
#include <Windows.h>
BYTE GDT[6] = {0};
DWORD dwH2GValue = 0;
void __declspec(naked) GetRegisetr()
```

```
{
    __asm
    {
        pushad
        pushfd
        mov eax,0x8003f00c // 读取高2G地址
        mov ebx,[eax]
        mov dwH2GValue,ebx
        sgdt  GDT
        popfd
        popad

        retf
    }
}

void PrintRegisetr()
{
    DWORD  GDT_ADDR = *(PDWORD)(&GDT[2]);
    WORD   GDT_LIMIT = *(PWORD)(&GDT[0]);
    printf("%x %x %x",dwH2GValue, GDT_ADDR, GDT_LIMIT)
}

int main()
{
    char buf[6];
    *(DWORD*) &buf[0] = 0x12345678;
    *(WORD*) &buf[4] = 0x48;
    __asm
    {
        call fword ptr[buf]
    }
    PrintRegisetr();
    getchar();
    return 0;
}
```

再次查找GetRegisetr的地址为00401040，设置调用门

```
eq 8003f048 0040EC00`00081040
```

执行代码

```
#include <iostream>
#include <Windows.h>
BYTE GDT[6] = {0};
DWORD dwH2GValue = 0;
void __declspec(naked) GetRegisetr()
{
    __asm
    {
        pushad
        pushfd
        mov eax, 0x8003f00c // 读取高2G地址
        mov ebx, [eax]
        mov dwH2GValue, ebx
        sgdt GDT
        popfd
        popad

        retf
    }
}

void PrintRegisetr()
{
    DWORD GDT_ADDR = *(PDWORD)(GDT[2]);
    WORD GDT_LIMIT = *(PWORD)(GDT[0]);
    printf("%x %x %x", dwH2GValue, GDT_ADDR, GDT_LIMIT);
}

int main()
{
    char buf[6];
    *(DWORD*) &buf[0] = 0x12345678;
    *(WORD*) &buf[4] = 0x48;
    __asm
    {
        call fword ptr[buf]
    }
    PrintRegisetr();
    getchar();
    return 0;
}
```

再对照Windbg里0x8003f00c地址处的内容

```
kd> dq 8003f000
8003f000  00000000`00000000 00cf9b00`0000ffff
8003f010  00cf9300`0000ffff 00cffb00`0000ffff
8003f020  00cff300`0000ffff 80008b04`200020ab
8003f030  ffc093df`f0000001 0040f300`00000fff
8003f040  0000f200`0400ffff 00000000`00000000
8003f050  80008954`af000068 80008954`af680068
8003f060  00009302`2f40ffff 0000920b`80003fff
8003f070  ff0092ff`700003ff 80009a40`0000ffff
kd> eq 8003f048 0040EC00`00081040
kd> g
```

可以看到我们成功在用户代码区获取了高2G内存的数据

2. 实现调用门(有参提权)

和上面的做法差不多，只需要**修改调用门的ParamCount(传入参数个数)**即可，直接给代码

```
eq 8003f048 0040EC03`00081030
```

```
#include <windows.h>
#include <stdio.h>
DWORD x = 0;
DWORD y = 0;
```



```

DWORD z = 0;
void __declspec(naked) Func()
{
    __asm
    {
        int 3;
        pushad;
        pushfd;
        mov eax,[esp+0x24+0x8+0x8];
        mov dword ptr ds:[x],eax;
        mov eax,[esp+0x24+0x8+0x4];
        mov dword ptr ds:[y],eax;
        mov eax,[esp+0x24+0x8];
        mov dword ptr ds:[z],eax;
    }

    __asm
    {
        popfd;
        popad;
        retf 0xc; // push了3个参数, 需要手动平衡堆栈
    }
}

void PrintRegisetr()
{
    printf("%x %x %x",x, y, z);
}

int main()
{
    char buf[6];
    *(DWORD*) &buf[0] = 0x12345678;
    *(WORD*) &buf[4] = 0x48;
    __asm
    {
        push 3;
        push 2;
        push 1;
        call fword ptr [buf];
    }
    PrintRegisetr();
    return 0;
}

```

Virtual: b1782dc4

b1782dc4 0040106e 0000001b 00000001 00000002 00000003 0012ff20 00000023

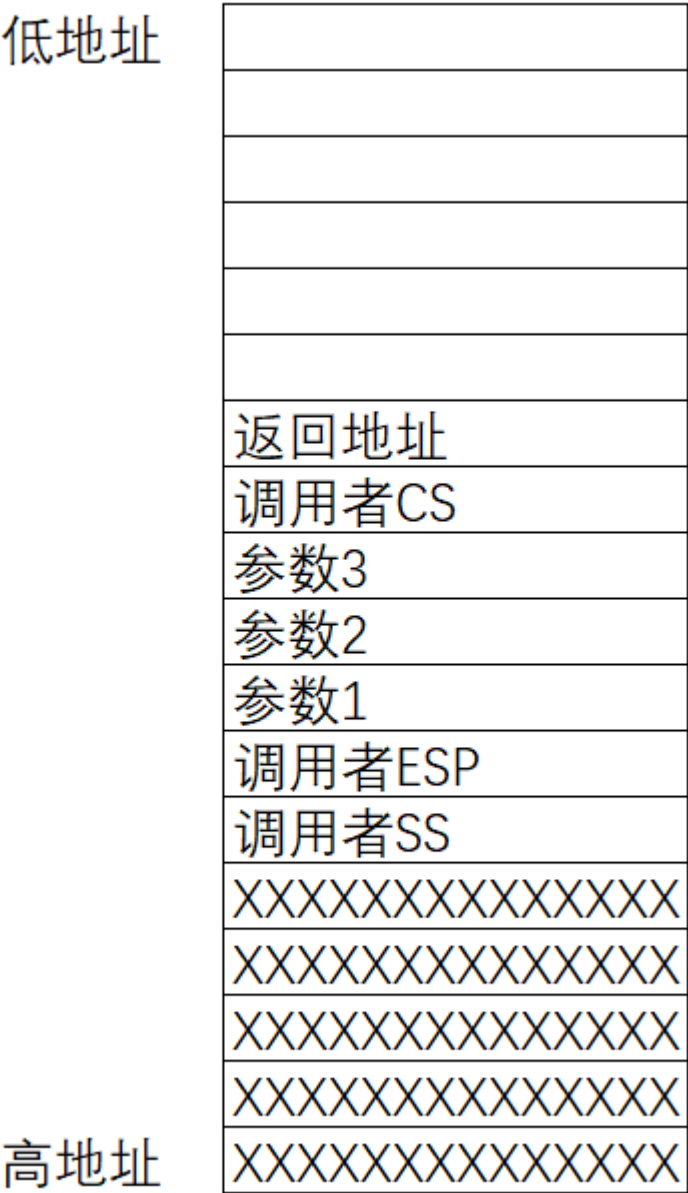
伪代码

```

push SS
push ESP
push 3

```

```
push 2
push 1
push CS
push 返回地址
```



调用printf报错C05

我检查了寄存器执行前后的变化，如图

```
EAX = CCCCCCCC EBX = 7FFDF000 ECX = 00000000 EDX = 00380D58 ESI = 00000000 EDI = 0012FF80
EIP = 0040107F ESP = 0012FF28 EBP = 0012FF80 EFL = 00000212 MM0 = 0000000000000000
MM1 = 0000000000000000 MM2 = 0000000000000000 MM3 = 0000000000000000 MM4 = 0000000000000000
MM5 = 0000000000000000 MM6 = 0000000000000000 MM7 = 0000000000000000
XMM0 = 00000000000000000000000000000000 XMM1 = 00000000000000000000000000000000
XMM2 = 00000000000000000000000000000000 XMM3 = 00000000000000000000000000000000
XMM4 = 00000000000000000000000000000000 XMM5 = 00000000000000000000000000000000
XMM6 = 00000000000000000000000000000000 XMM7 = 00000000000000000000000000000000 CS = 001B
DS = 0023 ES = 0023 SS = 0023 FS = 003B GS = 0000 OV=0 UP=0 EI=1 PL=0 ZR=0 AC=1 PE=0 CY=0
```

```
EAX = CCCCCCCC EBX = 7FFDF000 ECX = 00000000 EDX = 00380D58 ESI = 00000000 EDI = 0012FF80
EIP = 0040108E ESP = 0012FF28 EBP = 0012FF80 EFL = 00000212 MM0 = 0000000000000000
MM1 = 0000000000000000 MM2 = 0000000000000000 MM3 = 0000000000000000 MM4 = 0000000000000000
MM5 = 0000000000000000 MM6 = 0000000000000000 MM7 = 0000000000000000
XMM0 = 00000000000000000000000000000000 XMM1 = 00000000000000000000000000000000
XMM2 = 00000000000000000000000000000000 XMM3 = 00000000000000000000000000000000
XMM4 = 00000000000000000000000000000000 XMM5 = 00000000000000000000000000000000
XMM6 = 00000000000000000000000000000000 XMM7 = 00000000000000000000000000000000 CS = 001B
DS = 0023 ES = 0023 SS = 0023 FS = 0000 GS = 0000 OV=0 UP=0 EI=1 PL=0 ZR=0 AC=1 PE=0 CY=0
```

可以看到FS从3B变为了00，这是不正确的，查找资料发现

构造用门描述符前后堆栈变化中 FS从3B变成了30，是由于Func() 中INT 3 造成的，INT3会改变FS值

在线上中提出了一些问题，我若是在call里修改了ebp,eax,ecx等寄存器，返回后会恢复原来的寄存器还是为我在0环修改的内容，经过实验，发现返回后是我在0环修改的内容，所以需要在call内部手动保存寄存器

小测

需求

1、构造一个调用门，实现3环读取高2G内存。 2、在第一题的基础上进行修改，实现通过翻墙的方式返回到其他地址。 3、在第一题的基础上进行修改，在门中再建一个门跳转到其他地址。 工 要求: 代码正常执行不蓝屏

实现

1. 构造一个调用门，实现3环读取高2G内存。

调用门描述符

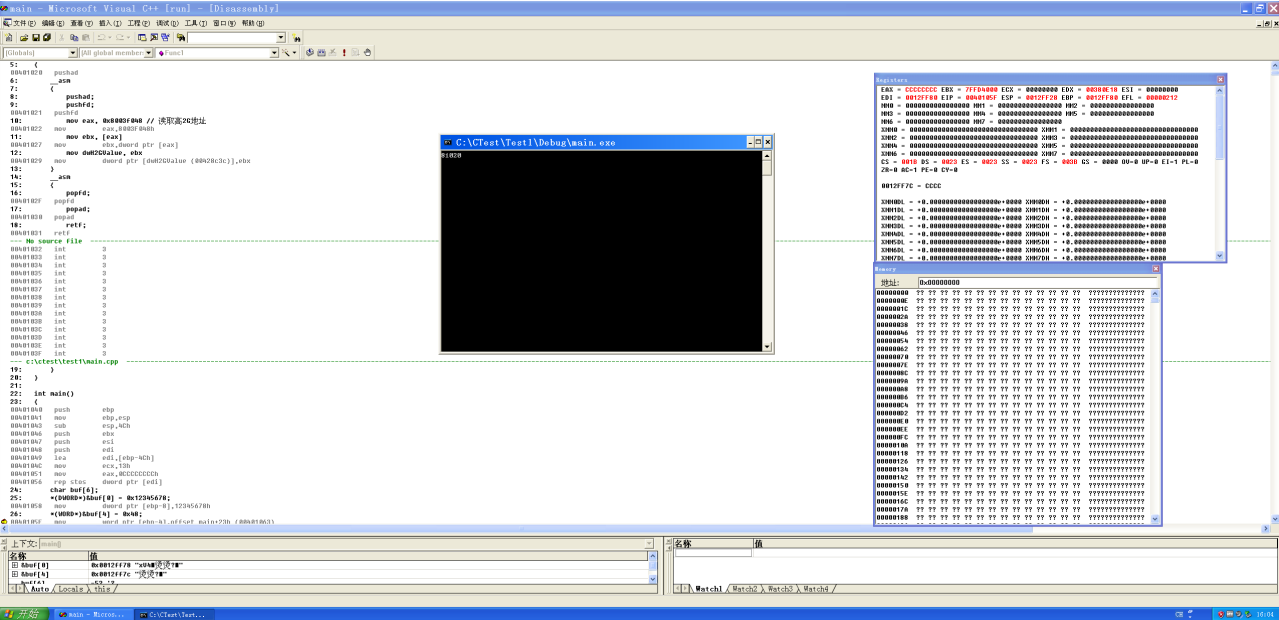
```
eq 8003f048 0040EC00`00081020
```

◦ 代码

```
#include <windows.h>
#include <stdio.h>
DWORD dwH2GValue = 0;
void __declspec(naked) Func1()
{
    __asm
    {
        pushad;
        pushfd;
        mov eax, 0x8003f048 // 读取高2G地址
        mov ebx, [eax]
        mov dwH2GValue, ebx
    }
    __asm
    {
        popfd;
    }
}
```

```
        popad;
        retf;
    }
}

int main()
{
    char buf[6];
    *(DWORD*)&buf[0] = 0x12345678;
    *(WORD*)&buf[4] = 0x48;
    __asm
    {
        call fword ptr[buf];
    }
    printf("%x", dwH2GValue);
    return 0;
}
```



2. 在第一题的基础上进行修改，实现通过翻墙的方式返回到其他地址

调用门描述符

```
eq 8003f048 0040EC00`00081030
```

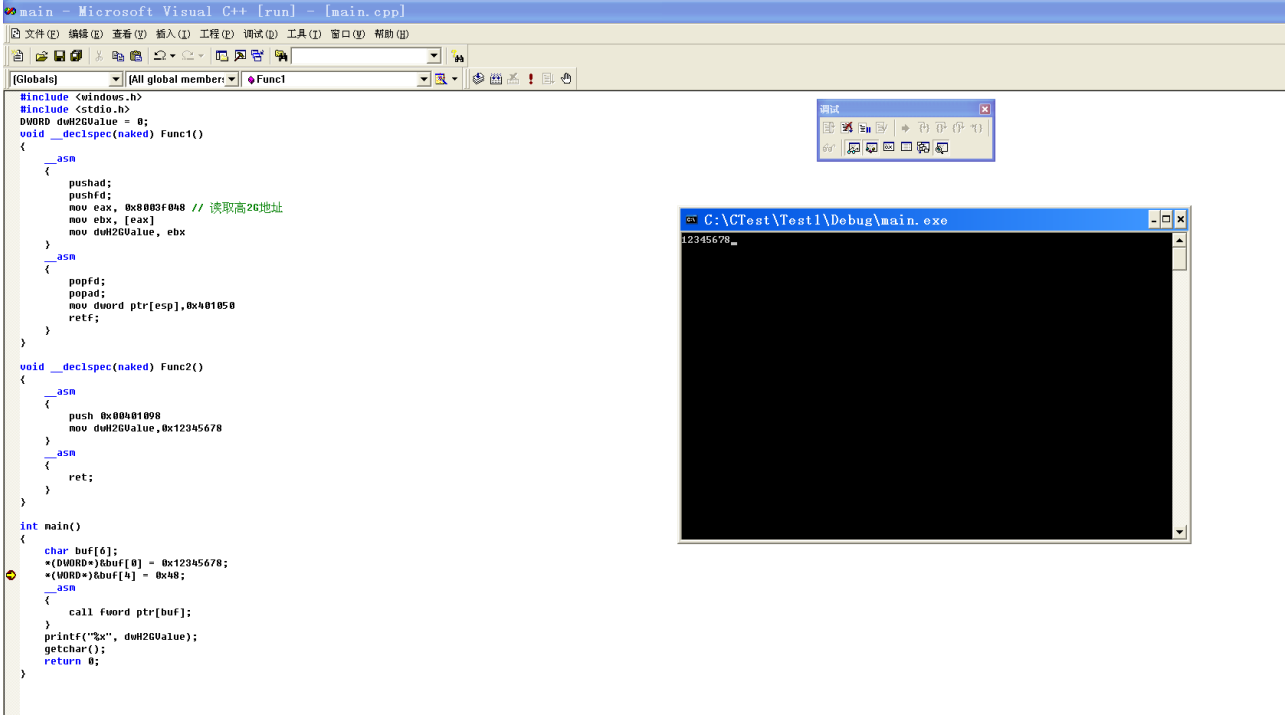
◦ 代码

```
#include <windows.h>
#include <stdio.h>
DWORD dwH2GValue = 0;
void __declspec(naked) Func1()
{
    __asm
```

```
{
    pushad;
    pushfd;
    mov eax, 0x8003f048 // 读取高2G地址
    mov ebx, [eax]
    mov dwH2GValue, ebx
}
__asm
{
    popfd;
    popad;
    mov dword ptr[esp], 0x401050 // 跳转到Func2的地址
    retf;
}
}

void __declspec(naked) Func2()
{
    __asm
    {
        push 0x00401098 // 返回main函数的地址
        mov dwH2GValue, 0x12345678
    }
    __asm
    {
        ret;
    }
}

int main()
{
    char buf[6];
    *(DWORD*)&buf[0] = 0x12345678;
    *(WORD*)&buf[4] = 0x48;
    __asm
    {
        call fword ptr[buf];
    }
    printf("%x", dwH2GValue);
    return 0;
}
```



3. 在第一题的基础上进行修改，在门中再建一个门跳转到其他地址

找到两个能写调用门描述符的地方

```
kd> dq 8003f000 L40
8003f000 00000000`00000000 00cf9b00`0000ffff
8003f010 00cf9300`0000ffff 00cffb00`0000ffff
8003f020 00cff300`0000ffff 80008b04`200020ab
8003f030 ffc093df`f0000001 0040f300`00000fff
8003f040 0000f200`0400ffff 0040ec00`00081030
8003f050 80008955`27000068 80008955`27680068
8003f060 00009302`2f40ffff 0000920b`80003fff
8003f070 ff0092ff`700003ff 80009a40`0000ffff
8003f080 80009240`0000ffff 00009200`00000000
8003f090 00000000`00000000 00000000`00000000
8003f0a0 8a008912`32880068 00000000`00000000
8003f0b0 00000000`00000000 00000000`00000000
8003f0c0 00000000`00000000 00000000`00000000
8003f0d0 00000000`00000000 00000000`00000000
8003f0e0 ba009f92`8000ffff 00009200`0000ffff
8003f0f0 8000984f`cfd003b7 00009200`0000ffff
8003f100 ba40935b`d400ffff ba40935b`d400ffff
8003f110 ba40935b`d400ffff 00000000`8003f120
8003f120 00000000`8003f128 00000000`8003f130
8003f130 00000000`8003f138 00000000`8003f140
8003f140 00000000`8003f148 00000000`8003f150
8003f150 00000000`8003f158 00000000`8003f160
8003f160 00000000`8003f168 00000000`8003f170
8003f170 00000000`8003f178 00000000`8003f180
8003f180 00000000`8003f188 00000000`8003f190
8003f190 00000000`8003f198 00000000`8003f1a0
8003f1a0 00000000`8003f1a8 00000000`8003f1b0
8003f1b0 00000000`8003f1b8 00000000`8003f1c0
8003f1c0 00000000`8003f1c8 00000000`8003f1d0
8003f1d0 00000000`8003f1d8 00000000`8003f1e0
8003f1e0 00000000`8003f1e8 00000000`8003f1f0
8003f1f0 00000000`8003f1f8 00000000`8003f200
```

一个是原先的0x8003f048

另一个是0x8003f090

调用门描述符

```
eq 0x8003f048 0040EC00`00081030
eq 0x8003f090 0040EC00`00081050
```

- 代码

```
#include <windows.h>
#include <stdio.h>
DWORD dwH2GValue = 0;
char buf1[6];
char buf2[6];
void __declspec(naked) Func1()
{
    __asm
    {
        pushad;
        pushfd;
        mov eax, 0x8003f048 // 读取高2G地址
        mov ebx, [eax]
        mov dwH2GValue, ebx
        __asm
        {
            call fword ptr[buf2];
        }
    }
    __asm
    {
        popfd;
        popad;
        retf;
    }
}

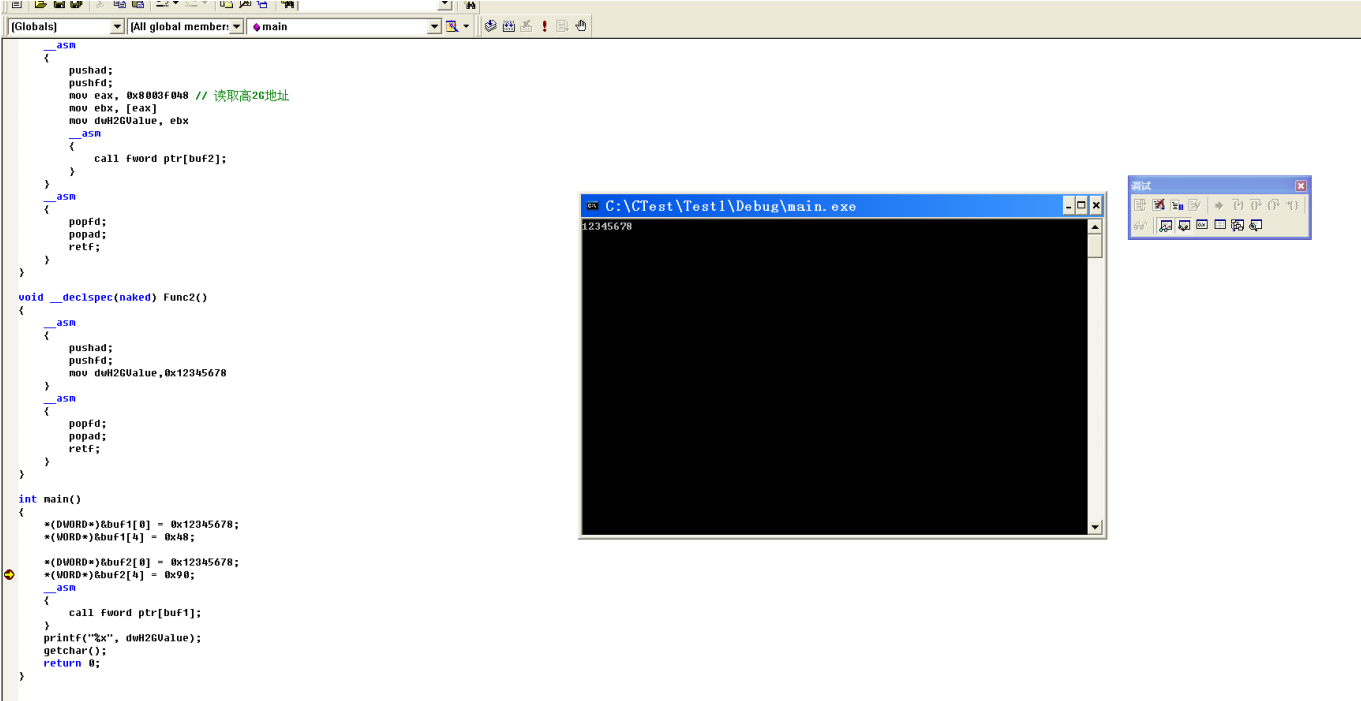
void __declspec(naked) Func2()
{
    __asm
    {
        pushad;
        pushfd;
        mov dwH2GValue, 0x12345678 // 重新赋值, 验证是否被执行
    }
    __asm
    {
        popfd;
        popad;
        retf;
    }
}

int main()
{
    *(DWORD*)&buf1[0] = 0x12345678;
    *(WORD*)&buf1[4] = 0x48;

    *(DWORD*)&buf2[0] = 0x12345678;
    *(WORD*)&buf2[4] = 0x90;
    __asm
```



```
{
    call fword ptr[buf1];
}
printf("%x", dwH2GValue);
return 0;
}
```

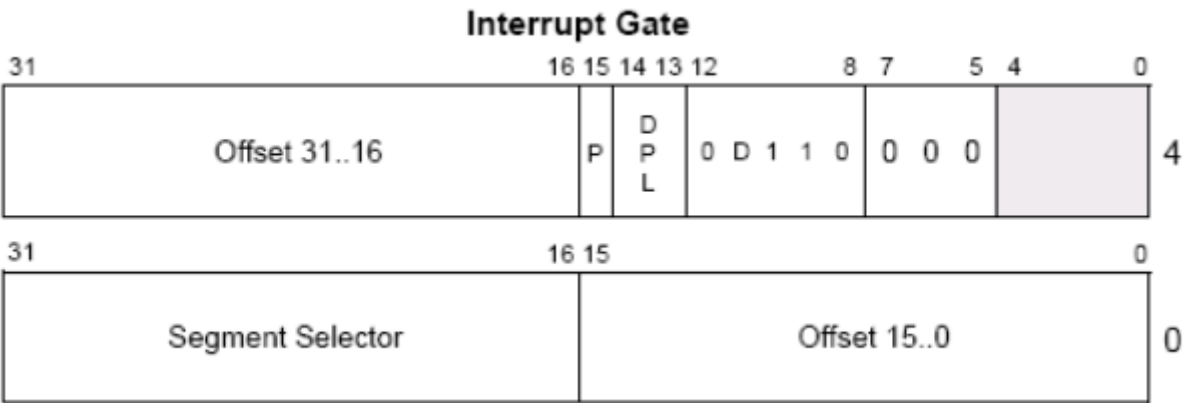


中断门

笔记

Table 3-2. System-Segment and Gate-Descriptor Types

Type Field					Description	
Decimal	11	10	9	8	32-Bit Mode	IA-32e Mode
0	0	0	0	0	Reserved	Upper 8 byte of an 16-byte descriptor
1	0	0	0	1	16-bit TSS (Available)	Reserved
2	0	0	1	0	LDT	LDT
3	0	0	1	1	16-bit TSS (Busy)	Reserved
4	0	1	0	0	16-bit Call Gate	Reserved
5	0	1	0	1	Task Gate	Reserved
6	0	1	1	0	16-bit Interrupt Gate	Reserved
7	0	1	1	1	16-bit Trap Gate	Reserved
8	1	0	0	0	Reserved	Reserved
9	1	0	0	1	32-bit TSS (Available)	64-bit TSS (Available)
10	1	0	1	0	Reserved	Reserved
11	1	0	1	1	32-bit TSS (Busy)	64-bit TSS (Busy)
12	1	1	0	0	32-bit Call Gate	64-bit Call Gate
13	1	1	0	1	Reserved	Reserved
14	1	1	1	0	32-bit Interrupt Gate	64-bit Interrupt Gate
15	1	1	1	1	32-bit Trap Gate	64-bit Trap Gate



• 与调用门的区别

数据位	31-16	15	14-13	12	11-8	7-5	4-0
含义	offset	P	DPL	S	Type	无	param.count
调用门	偏移	有效位	特权等级	值为0	值为1100	值为000	可以传递参数
中断门	同上	同上	同上	同上	值为1110	值为000	不允许传参，固定为0000

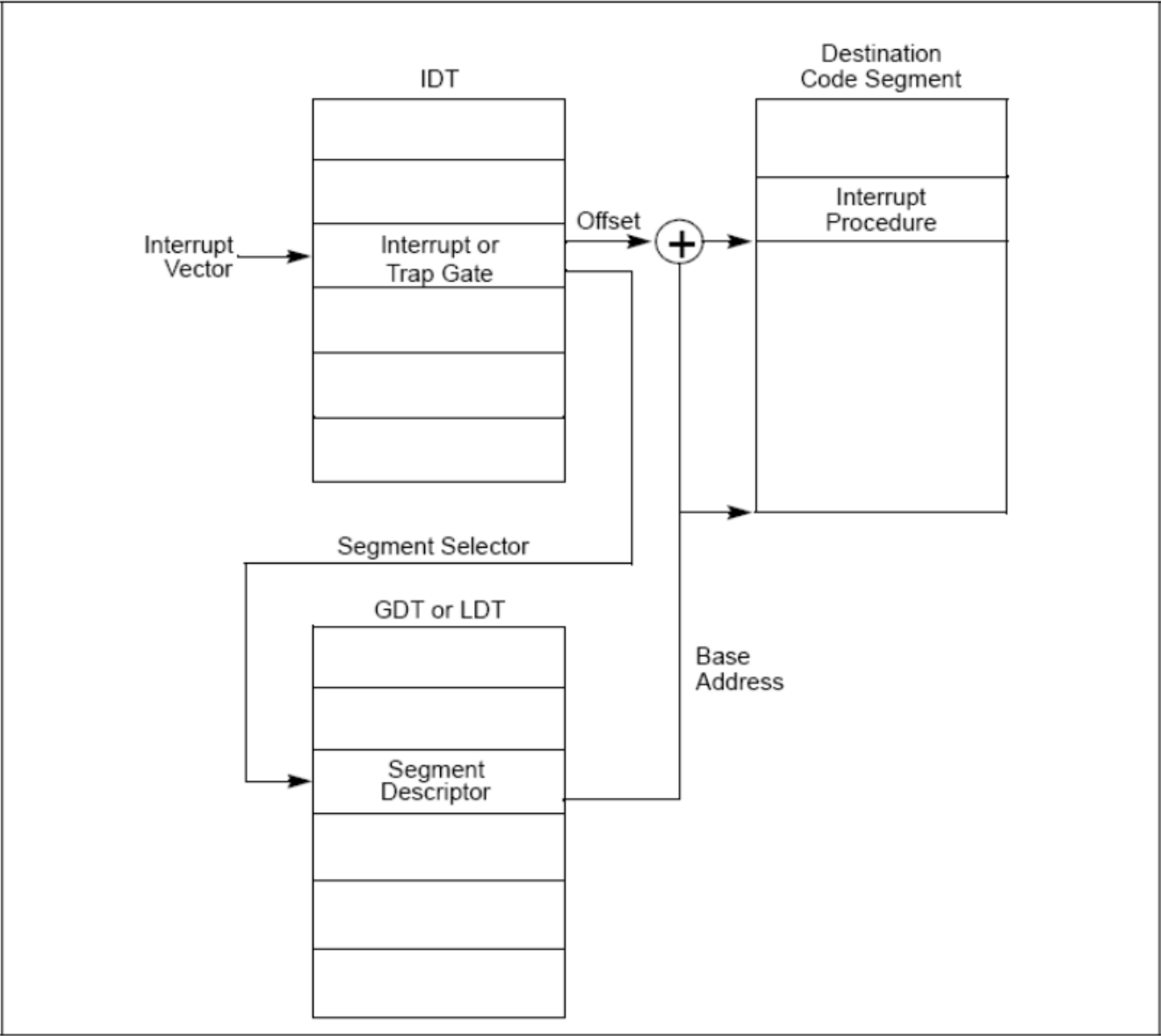


Figure 5-3. Interrupt Procedure Call

构造基础中断门描述符

数据位	31-16	15	14-13	12	11-8	7-5	4-0
含义	offset	P	DPL	S	Type	无	param
解释	段中偏移	有效位	特权等级	值为0	值为1110	值为000	不允许传参
值（二进制）	0	1	11	0	1110	000	0000

数据位 31-16 15-0 含义 selector offset 解释 段选择子 段中偏移 值（十六进制） 0x0008 0

```
0000EE00`00080000
```

课后练习

1. 实现中断门

2. 在调用门中使用iret返回，在中断门中使用retf返回

陷阱门

笔记

课后练习